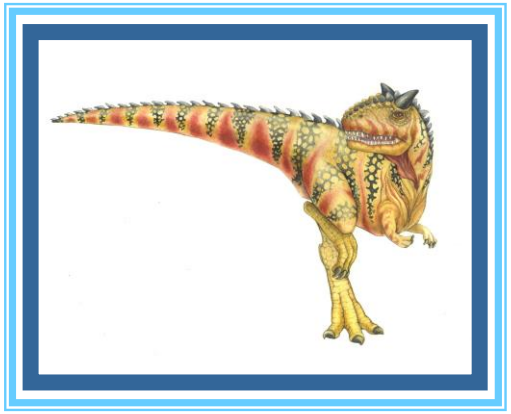
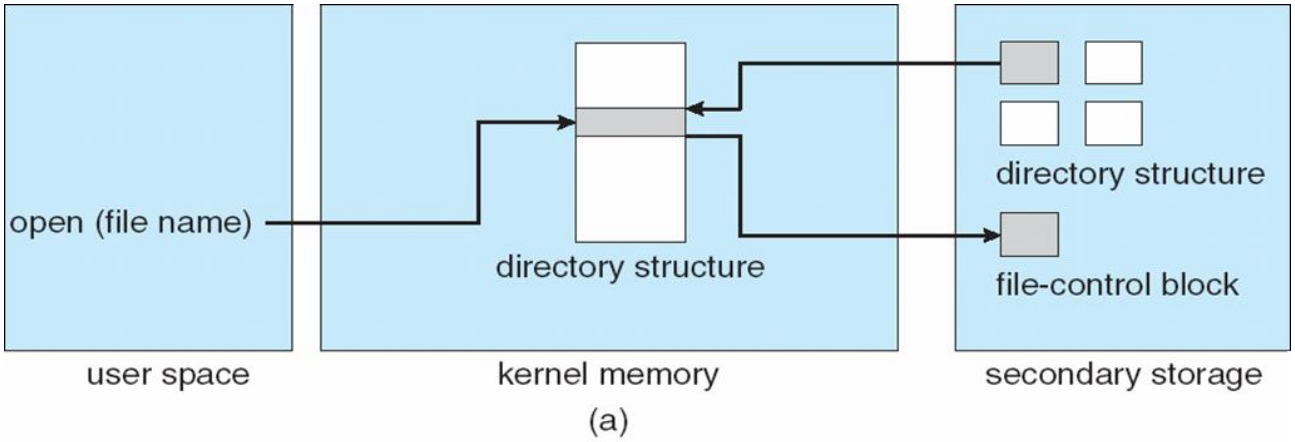
Supplement to Chapter 14: File System Implementation

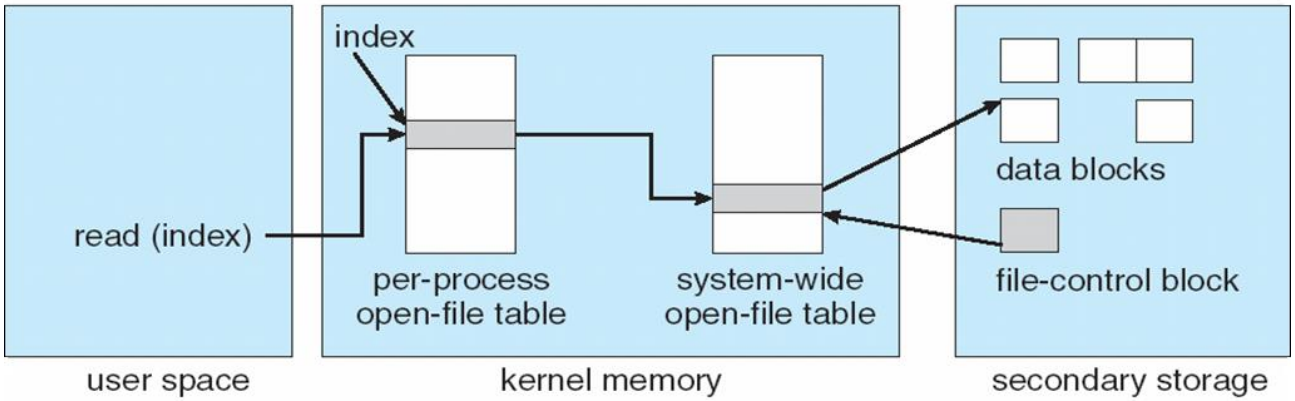


Silberschatz, Galvin and Gagne ©2018

In-Memory File System Structures

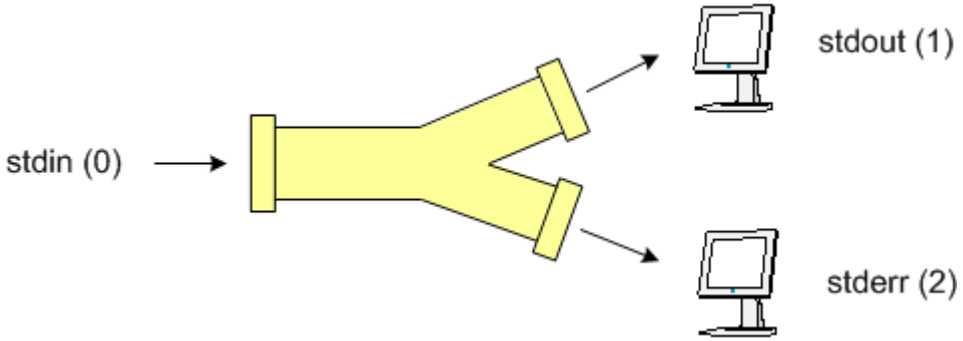


(a)



(b)

Example of I/O Redirection



In typical Unix installations, input (by default) comes from the terminal and the resulting output is displayed on the monitor.

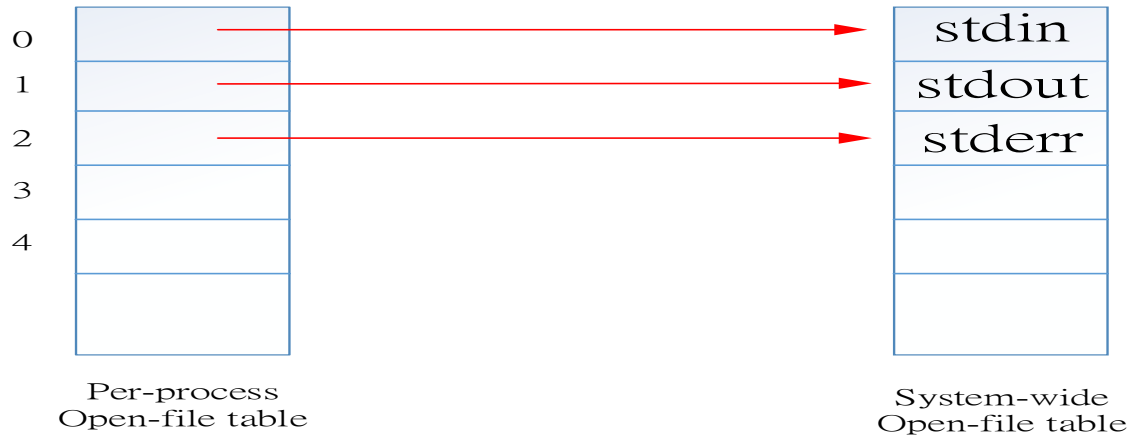
fd=0 – stdin fd=1 – stdout fd=2 – stderr

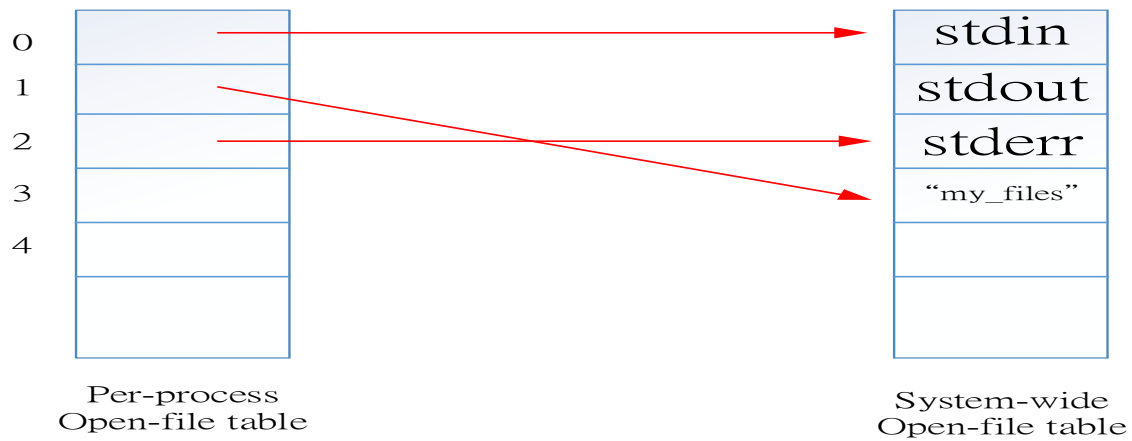
```
$ echo "Hello World!" [Enter]
Hello World!
$ echo "Hello World!" > my_files [Enter]
$ cat my_files [Enter]
Hello World!
```

Example of I/O Redirection

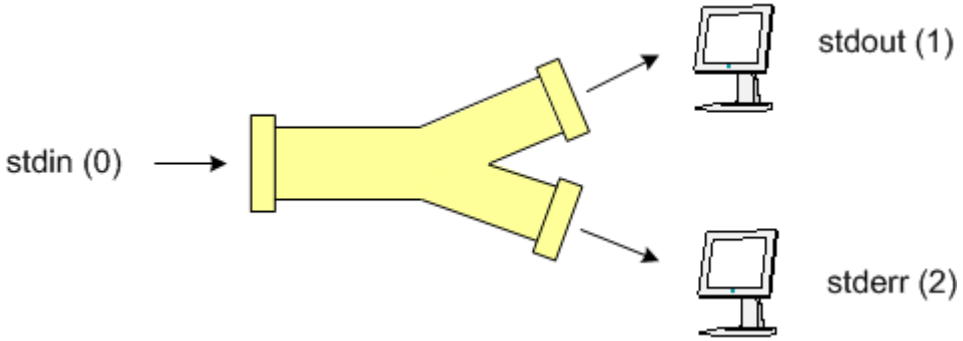
$ echo "Hello World!" [Enter]

$ echo "Hello World!" > my_files [Enter]





Example of I/O Redirection



$ wc my_files [Enter]

6 7 39 my_files

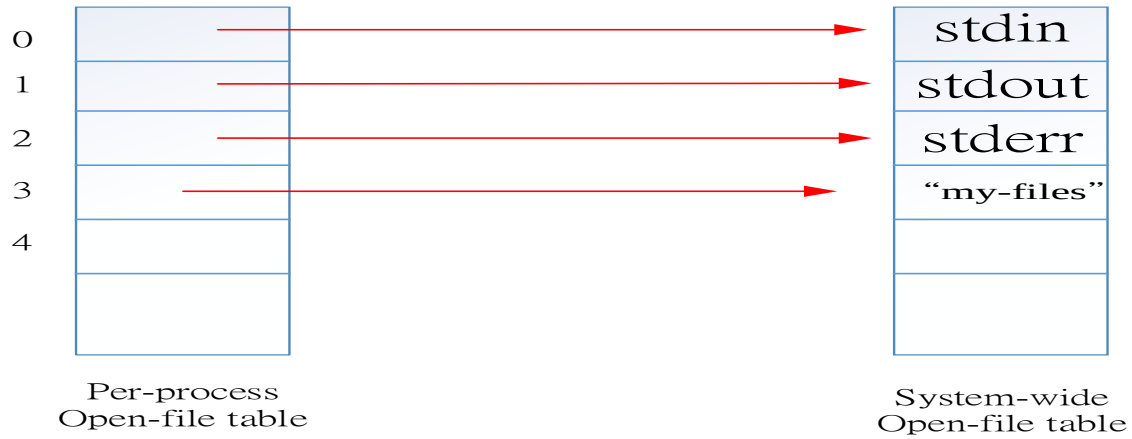
$ wc < my_files [Enter]

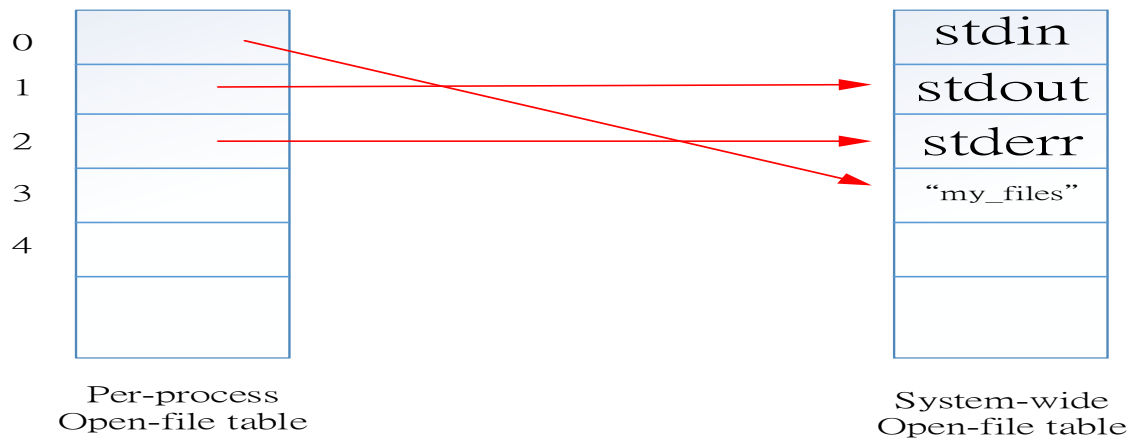
6 7 39

Example of I/O Redirection

$ wc my_files [Enter]

$ wc < my_files [Enter]

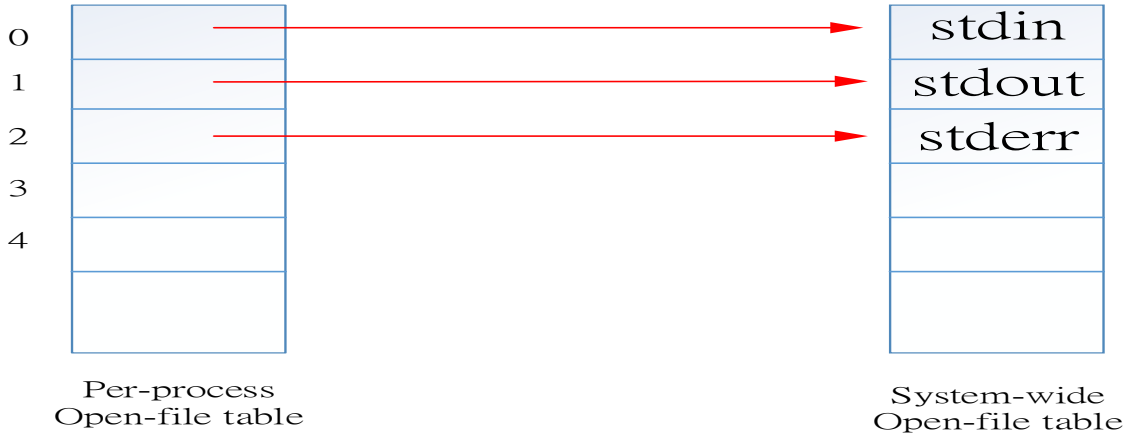


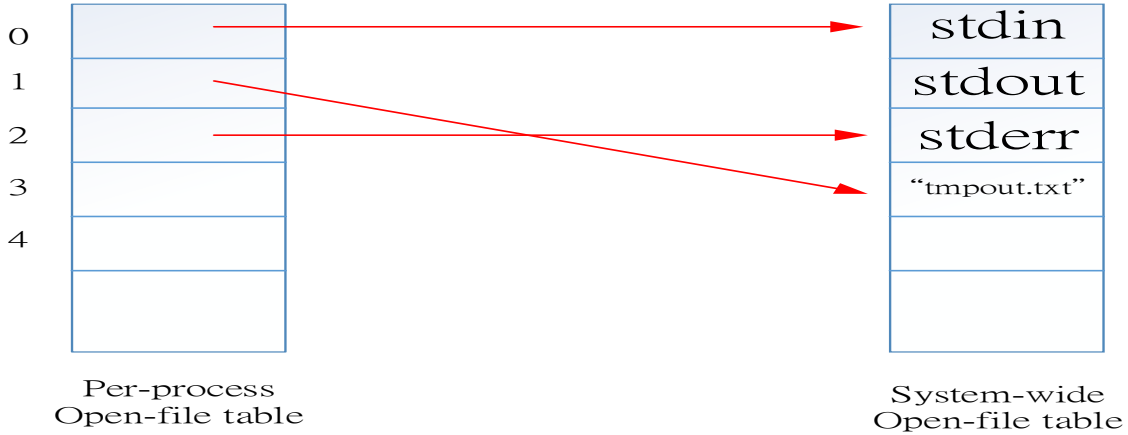


Example of I/O Redirection

$ ps –aux [Enter]

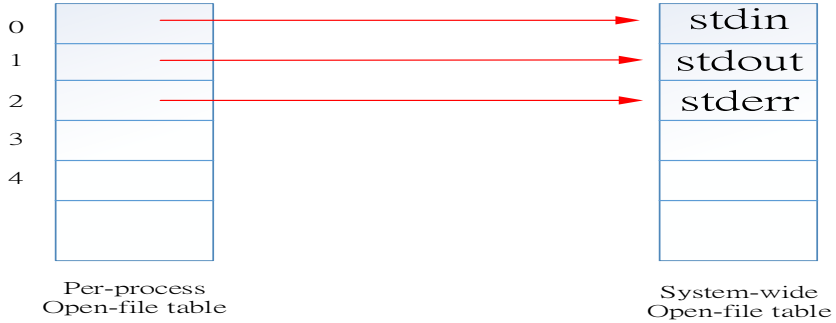
$ ps –aux > tmpout.txt [Enter]

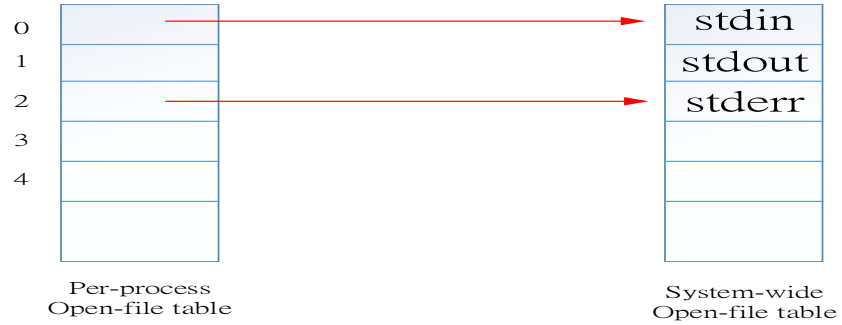




Example of I/O Redirection

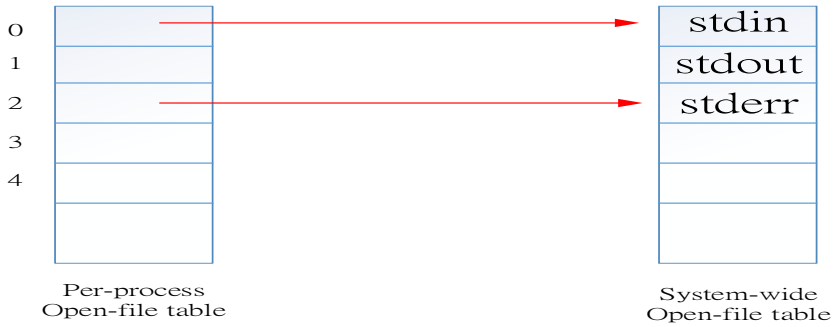
```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>

int main(void)
{
    printf("entering main process---\n");
    int ret;
    int fd;
    char *argv[] = {"ps","-aux",NULL};

    close(1);
    fd=open("tmpout.txt",O_WRONLY | O_CREAT);
    ret = execvp("ps",argv);
    if(ret == -1)
        perror("execl error");
    printf("exiting main process ----\n");
    return 0;
}
```

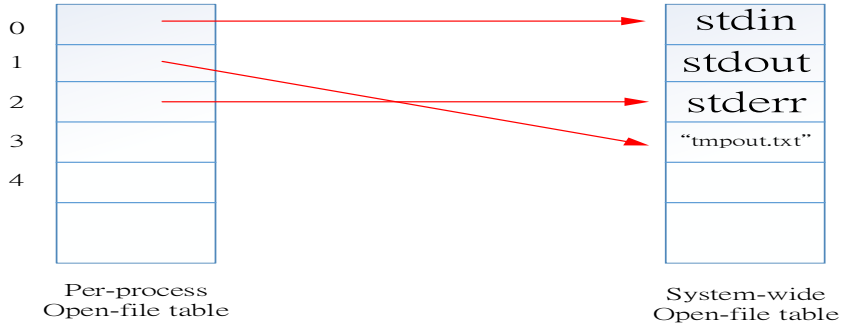




Example of I/O Redirection

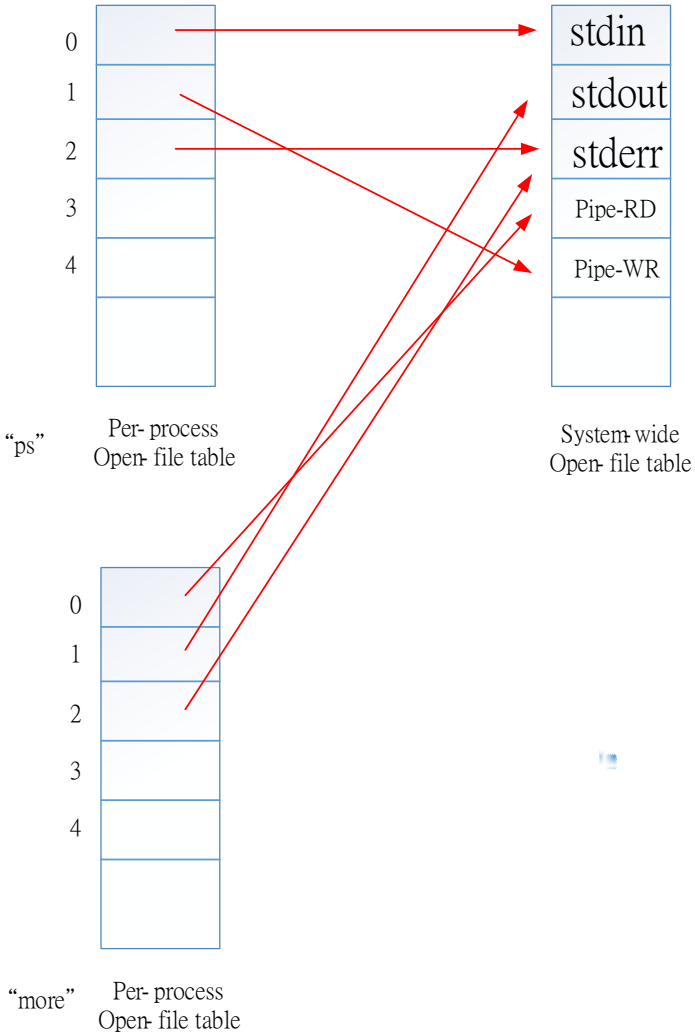
```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>

int main(void)
{
    printf("entering main process---\n");
    int ret;
    int fd;
    char *argv[] = {"ps","-aux",NULL};

    close(1);
    fd=open("tmpout.txt",O_WRONLY | O_CREAT);
    ret = execvp("ps",argv);
    if(ret == -1)
        perror("execl error");
    printf("exiting main process ----\n");
    return 0;
}
```





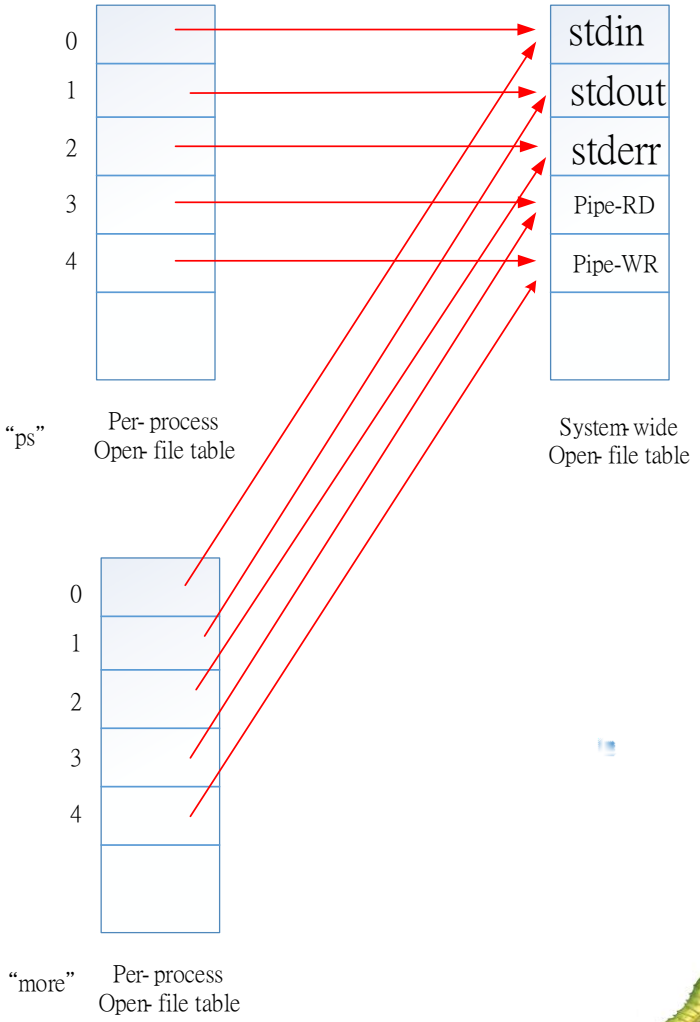
Example of I/O Redirection

$ ps –aux | more [Enter]



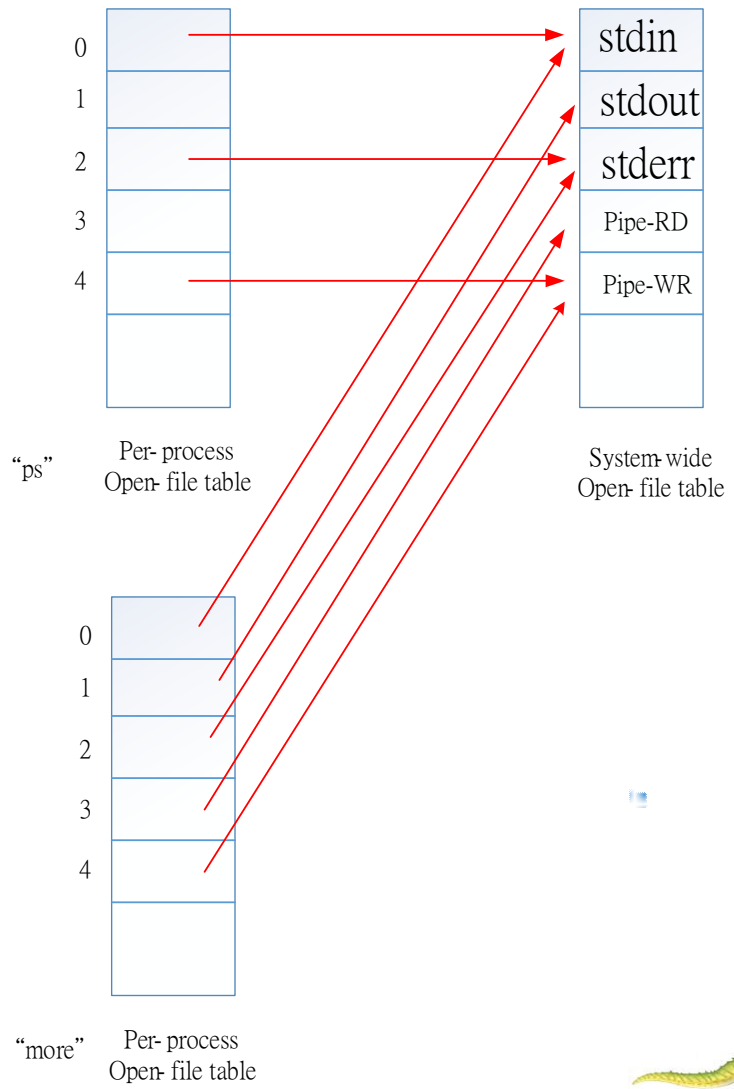
Example of I/O Redirection

```
int main(void)
{  int fd[2];     pid_t pid2;     int ret0, ret1;
   char *argv0[] = {"ps", "-aux", NULL};
   char *argv1[] = {"more", NULL};
   if(pipe(fd) == -1) {
      fprintf(stderr, "Pipe failed");
      return 1;     }
   pid2 = fork();
   if(pid2 < 0) {
      fprintf(stderr, "Fork failed");
      return 1;     }
   if(pid2 == 0) {   /*child process */
      close(fd[0]);
      close(1);
      dup(fd[1]);
      close(fd[1]);
      ret0 = execvp("ps",argv0);
   } else { /* parent process */
      close(fd[1]);
      close(0);
      dup(fd[0]);
      close(fd[0]);
      ret1 = execvp("more",argv1);
   }
   return 0;
}
```



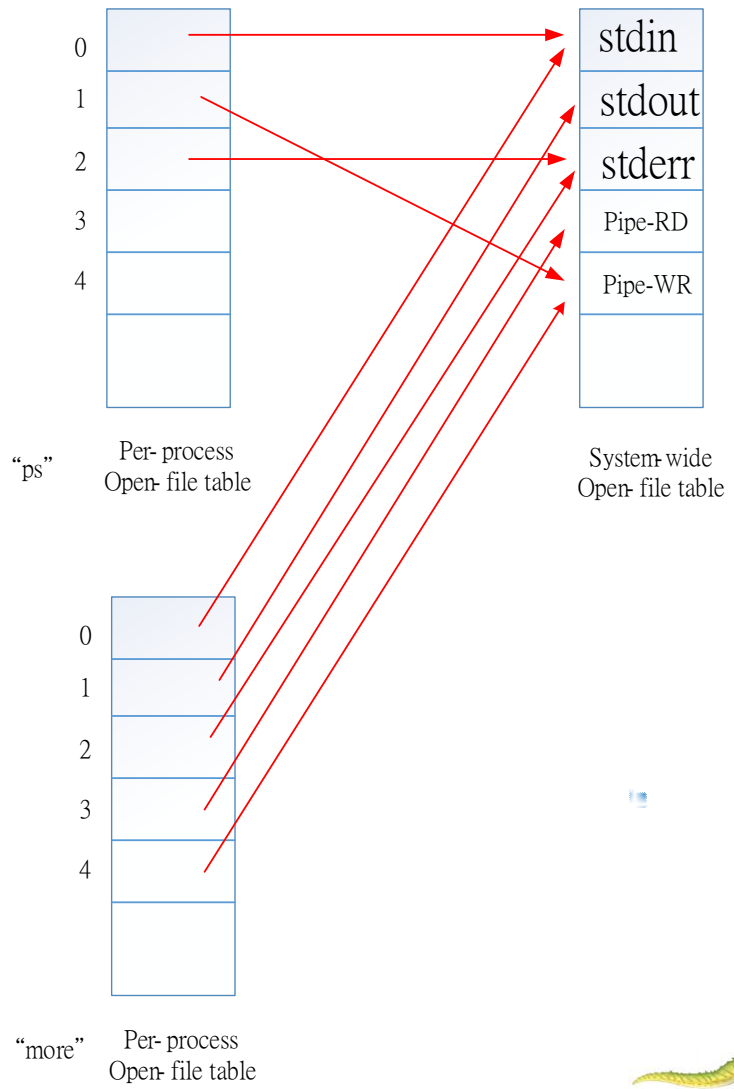
Example of I/O Redirection

```
int main(void)
{  int fd[2];     pid_t pid2;     int ret0, ret1;
   char *argv0[] = {"ps", "-aux", NULL};
   char *argv1[] = {"more", NULL};
   if(pipe(fd) == -1) {
      fprintf(stderr, "Pipe failed");
      return 1;     }
   pid2 = fork();
   if(pid2 < 0) {
      fprintf(stderr, "Fork failed");
      return 1;     }
   if(pid2 == 0) {   /*child process */
      close(fd[0]);
      close(1);
      dup(fd[1]);
      close(fd[1]);
      ret0 = execvp("ps",argv0);
   } else { /* parent process */
      close(fd[1]);
      close(0);
      dup(fd[0]);
      close(fd[0]);
      ret1 = execvp("more",argv1);
   }
   return 0;
}
```



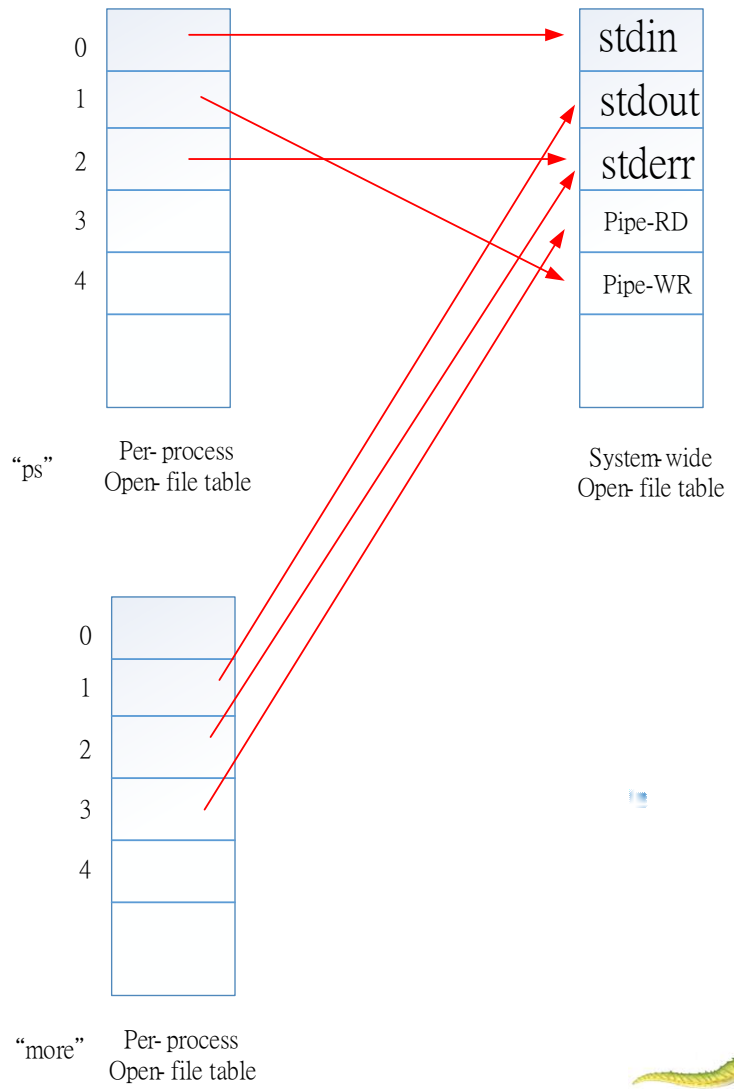
Example of I/O Redirection

```
int main(void)
{  int fd[2];     pid_t pid2;     int ret0, ret1;
   char *argv0[] = {"ps", "-aux", NULL};
   char *argv1[] = {"more", NULL};
   if(pipe(fd) == -1) {
      fprintf(stderr, "Pipe failed");
      return 1;     }
   pid2 = fork();
   if(pid2 < 0) {
      fprintf(stderr, "Fork failed");
      return 1;     }
   if(pid2 == 0) {   /*child process */
      close(fd[0]);
      close(1);
      dup(fd[1]);
      close(fd[1]);
      ret0 = execvp("ps",argv0);
   } else { /* parent process */
      close(fd[1]);
      close(0);
      dup(fd[0]);
      close(fd[0]);
      ret1 = execvp("more",argv1);
   }
   return 0;
}
```



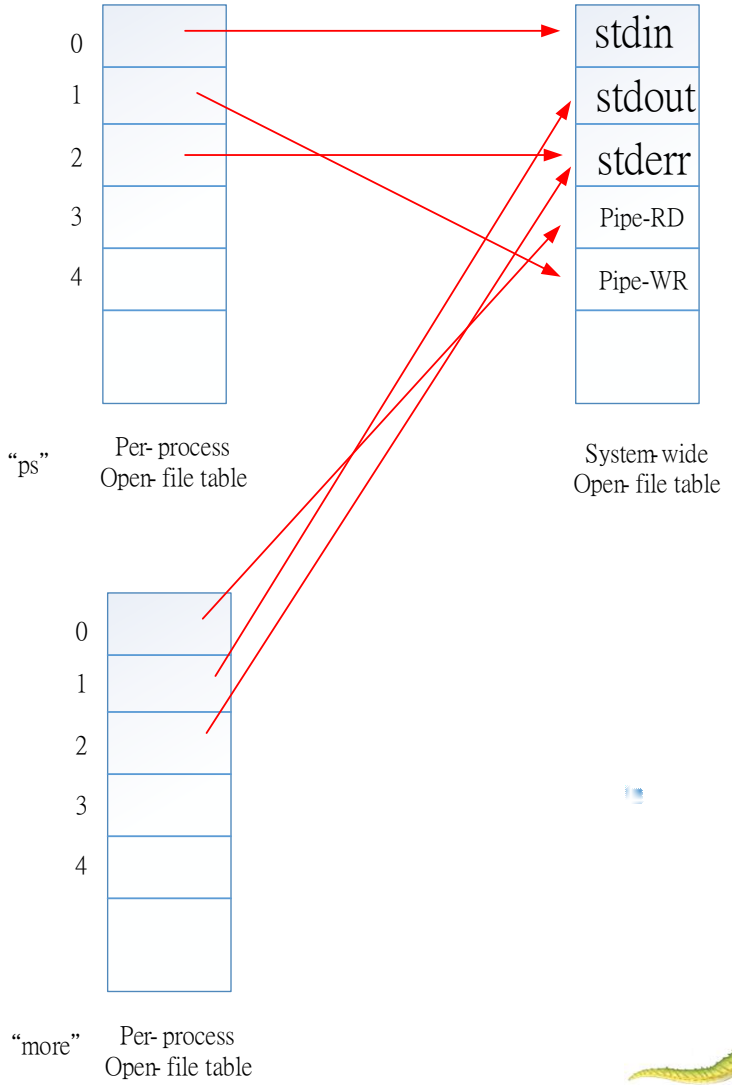
Example of I/O Redirection

```
int main(void)
{  int fd[2];     pid_t pid2;     int ret0, ret1;
   char *argv0[] = {"ps", "-aux", NULL};
   char *argv1[] = {"more", NULL};
   if(pipe(fd) == -1) {
      fprintf(stderr, "Pipe failed");
      return 1;     }
   pid2 = fork();
   if(pid2 < 0) {
      fprintf(stderr, "Fork failed");
      return 1;     }
   if(pid2 == 0) {   /*child process */
      close(fd[0]);
      close(1);
      dup(fd[1]);
      close(fd[1]);
      ret0 = execvp("ps",argv0);
   } else { /* parent process */
      close(fd[1]);
      close(0);
      dup(fd[0]);
      close(fd[0]);
      ret1 = execvp("more",argv1);
   }
   return 0;
}
```

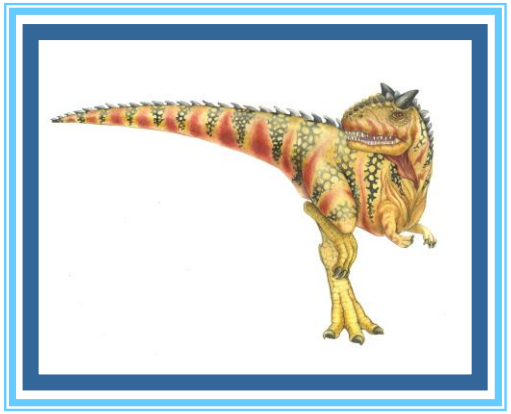


Example of I/O Redirection

```
int main(void)
{  int fd[2];     pid_t pid2;     int ret0, ret1;
   char *argv0[] = {"ps", "-aux", NULL};
   char *argv1[] = {"more", NULL};
   if(pipe(fd) == -1) {
      fprintf(stderr, "Pipe failed");
      return 1;     }
   pid2 = fork();
   if(pid2 < 0) {
      fprintf(stderr, "Fork failed");
      return 1;     }
   if(pid2 == 0) {   /*child process */
      close(fd[0]);
      close(1);
      dup(fd[1]);
      close(fd[1]);
      ret0 = execvp("ps",argv0);
   } else { /* parent process */
      close(fd[1]);
      close(0);
      dup(fd[0]);
      close(fd[0]);
      ret1 = execvp("more",argv1);
   }
   return 0;
}
```



End of Supplement of Chapter 14



Silberschatz, Galvin and Gagne ©2018